



Xiaomi AIoT 开发平台

桌面智能管家开发手册

手册版本： V1.0（全功能联调版本）

适用对象： 项目开发人员、二次开发开发者

文档依据： 产品设计方案与正式工程代码

目 录

一、手册概述.....	1
二、系统整体架构	1
2.1 硬件架构.....	1
2.2 软件整体架构.....	2
三、编译与部署指南	3
3.1 环境依赖.....	3
3.2 目录结构.....	3
3.3 编译步骤.....	5
3.4 部署与运行.....	5
四、核心模块开发说明	5
4.1 I2C 总线与设备探测子系统	5
4.2 传感器驱动模块	8
4.3 执行器驱动模块	9
4.4 E5 触控屏 Web 交互系统	10
4.5 音频与语音交互系统	11
4.6 智能联动引擎.....	13
4.7 网络监控与容错模块	15
五、接口定义与二次开发指南	15
5.1 新增 I2C 设备开发步骤	15
5.2 新增语音意图开发步骤	15
5.3 新增 Web API 开发步骤	15

5.4 新增智能联动场景开发步骤	16
六、接口定义与二次开发指南	16
6.1 xiaomi_config.json	16
6.2 运行时状态文件	17
七、常见问题与调试方法	17

一、手册概述

本手册详细描述桌面智能管家系统的硬件架构、软件分层设计、核心模块实现逻辑、编译部署流程、接口定义与二次开发规范，所有内容均基于现有工程代码与产品设计，可直接用于系统调试、功能扩展与维护。

系统基于 U2P-X3MAI 主控开发，采用“底层硬件驱动 + 核心业务逻辑 + 多端交互 + 云端 AI 赋能”的分层架构，支持语音、按键、触控屏多模交互，具备离线降级运行能力，可适配办公、居家等桌面场景。

二、系统整体架构

2.1 硬件架构

系统采用母板 + 子板的堆叠 / 平铺混合架构，所有外设通过 I2C 总线与主控通信，硬件配置如下表所示：

母板槽位	子板类型	安装方式	核心功能
槽 0	U2P-X3M	底层主控	系统主控、任务调度、网络通信、数据存储
槽 0（纵向）	E4	堆叠于 U2P 上方	音频功放、语音播报、状态提示
槽 0（纵向）	S3	堆叠于 E4 上方（最上层）	5 麦克风阵列、语音信号采集
槽 0（直连）	E5	直连 U2P	触控屏交互、Web 界面展示
槽 1	E1	平铺	4 位数码管显示、RGB 全彩状态灯
槽 2	S2	平铺	光照/温/湿度环境监测（BH1750 芯片、SHT3x 芯片）
槽 3	S7	平铺	人体存在检测、设备休眠 / 唤醒触发
槽 4	S1	平铺	12 键矩阵键盘、本地物理操控

主板槽位	子板类型	安装方式	核心功能
槽 5	E2	平铺	风扇 PWM 无级调速控制

表 1：硬件架构表

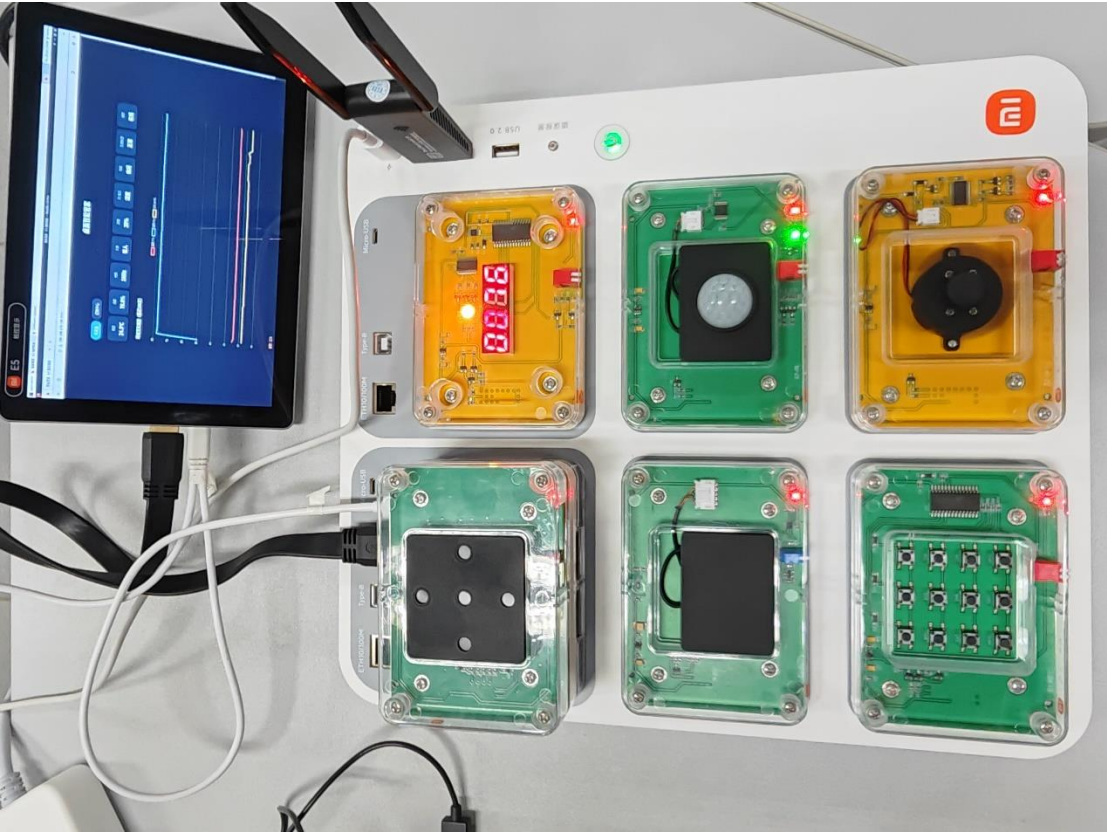


图 1：硬件整体堆叠装配示意图

2.2 软件整体架构

系统采用四层解耦设计，从上到下依次为 AI 能力层、交互服务层、核心业务层、硬件驱动层，各层通过标准化接口通信。

层级	包含模块	核心职责
AI 能力层	小米 ASR/NLP/TTS 接口、离线关键词匹配、真实天气查询	语音识别、语义理解、语音合成、扩展技能调用
交互服务层	E5 Web 服务、按键事件处理、音频播放线程、网络监控	多端交互指令接收、状态可视化展示、系统状态同步

层级	包含模块	核心职责
核心业务层	智能联动引擎、场景管理、休眠唤醒逻辑、语音交互流程控制	业务逻辑编排、场景化策略执行、全链路状态管理
硬件驱动层	I2C 总线驱动、设备自动探测、各传感器 / 执行器驱动	硬件寄存器操作、数据采集、执行器控制

表 2：软件整体架构表

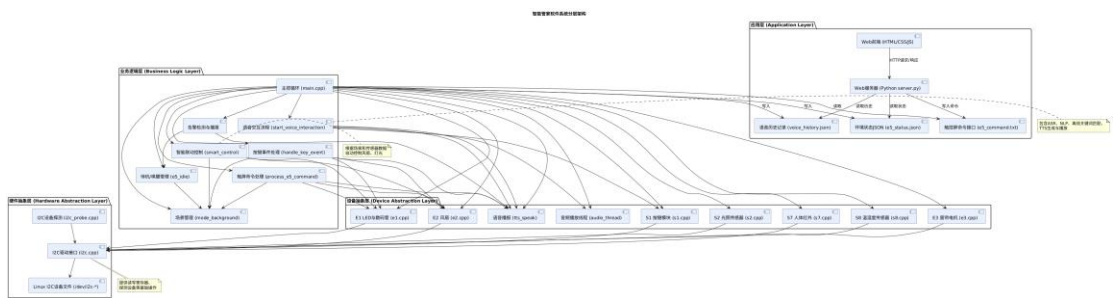


图 2：系统软件分层架构框图

三、编译与部署指南

3.1 环境依赖

- 硬件平台：U2P-X3M 开发板（Linux 操作系统）
- 编译工具：g++ 编译器、GNU Make
- 系统依赖：pthread 线程库、Python 3.x、tinycap/tinyplay/aplay 音频工具
- 前端依赖：Chart.js

3.2 目录结构

工程根目录包含以下核心目录与文件：

- inc/ # 头文件
 - i2c.h # I2C 底层通信
 - i2c_probe.h # I2C 设备探测
 - s1.h # S1 键盘
 - s2.h # S2 光照

—— s7.h	# S7 人体红外
—— s8.h	# S8 温湿度
—— e1.h	# E1 数码管+RGB
—— e2.h	# E2 风扇
—— src/	# 源文件
—— i2c.cpp	# I2C 通信实现
—— i2c_probe.cpp	# 设备探测实现
—— s1.cpp	# S1 键盘实现
—— s2.cpp	# S2 光照实现
—— s7.cpp	# S7 人体红外实现
—— s8.cpp	# S8 温湿度实现
—— e1.cpp	# E1 数码管+RGB 实现
—— e2.cpp	# E2 风扇实现
—— main.cpp	# 主程序
—— web/	# Web 前端+服务
—— index.html	# 主界面
—— styles.css	# 样式表
—— refresh.js	# 前端逻辑
—— server.py	# HTTP 服务器
—— xiaomi_ai.py	# AI 接口脚本
—— xiaomi_config.json	# AI 配置
—— run_server.sh	# 启动脚本
—— e5_status.json	# 状态数据（C++生成）
—— e5_command.txt	# 命令管道
—— voice_history.json	# 语音历史
—— Makefile	
—— README.md	

3.3 编译步骤

系统使用 Makefile 进行构建，执行以下命令完成编译：

```
01  # 清理旧编译产物
02  make clean
03
04  # 全量编译，生成可执行文件 smart_hub
05  make
```

编译参数说明：

编译选项：-Wall -O2 -g，开启警告、O2 优化、调试信息

链接选项：-lpthread，链接 POSIX 线程库

头文件路径：-I./inc

3.4 部署与运行

1. 将编译生成的 smart_hub 可执行文件与 web/目录整体拷贝到开发板指定路径

2. 配置小米 AI 平台密钥：编辑 web/xiaomi_config.json，填入有效 API_KEY

3. 启动 Web 服务：

```
01  cd web
02  chmod +x run_server.sh
03  ./run_server.sh 8000
```

4. 启动主程序：

```
01  ./smart_hub
```

5. 访问 E5 触控屏：在浏览器输入 http://开发板 IP:8000 即可进入控制界面

四、核心模块开发说明

4.1 I2C 总线与设备探测子系统

(1) 功能概述

统一管理所有 I2C 外设，实现总线自动扫描、设备地址自动识别、多同类型设备兼容，为上层驱动提供标准化的读写接口，无需手动配置硬件地址。

(2) 核心文件

头文件：i2c.h、i2c_probe.h

源文件：i2c.cpp、i2c_probe.cpp

(3) 核心实现逻辑

总线扫描范围：默认扫描/dev/i2c-4 至/dev/i2c-9 共 6 条 I2C 总线

设备探测机制：通过 SMBus 快速写命令检测地址响应，支持单个设备多个候选地址

多设备支持：单种子板最多支持 4 个同类型设备同时接入，地址存入 detected_addrs 数组

设备列表：全局数组 I2C_PROBE_LIST 预定义所有支持设备的名称、候选地址，目前支持 11 类设备

(4) 关键接口

接口函数	功能说明
<code>void i2c_probe(i2c_probe_target *list, int len)</code>	批量探测所有 I2C 设备，填充设备信息
<code>int i2c_device_idx(const char *name)</code>	根据设备名称查找索引
<code>int I2c_write_reg(int fd, unsigned char addr, unsigned char reg, unsigned char value)</code>	向指定设备寄存器写入单字节
<code>int I2c_read_regs(int fd, unsigned char addr, unsigned char reg, unsigned char *val, int num)</code>	从指定设备寄存器读取多字节

表 3：关键接口功能表

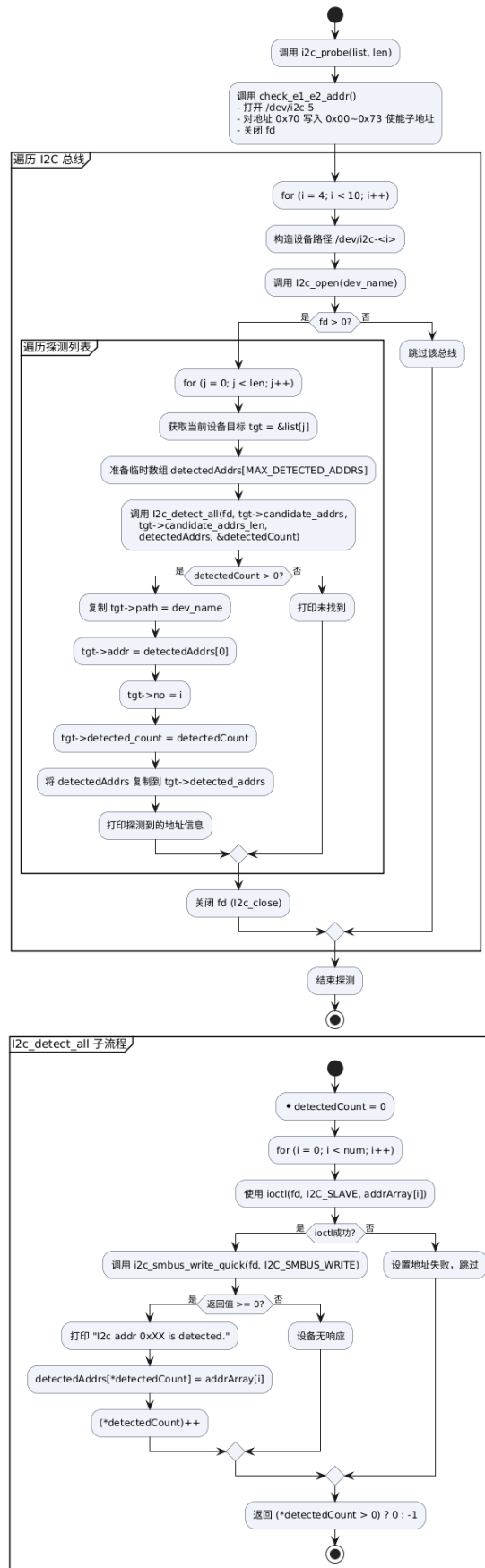


图 3: I2C 设备自动探测流程图

4.2 传感器驱动模块

4.2.1 S1 矩阵键盘驱动

功能：12 键矩阵输入，支持短按、长按双事件识别，硬件消抖 + 软件消抖
双重处理

核心数据结构：s1_key_process，包含按键状态机、按下时间戳、长短按标志位

状态机设计：

KEY_NULL：无按键空闲状态

KEY_PRESSED：按键按下待消抖状态

KEY_HOLDING：按键确认按下，持续计时

关键参数：消抖时间 20ms，短按判定 $\leq 600\text{ms}$ ，长按判定 $\geq 2000\text{ms}$

按键映射：SW1~SW9 对应数字键 1-9, SWA 对应 * 键, SW0 对应 0 键，
SWC 对应 #键

核心函数：s1_key_scan()（读取硬件键值）、process_keys()（状态机处理事件）



图 4: I2C 设备自动探测流程图

4.2.2 S2 光照温湿度传感器驱动

(1) 光照传感器驱动

芯片型号：BH1750

功能：实时采集环境光照强度，单位勒克斯（lx）

采样流程：发送测量指令→等待 120ms 转换时间→读取 2 字节原始数据→
换算为实际光照值

核心函数：s2_read_bh1750_value()

(2) 温湿度传感器驱动

芯片型号: SHT3x

功能: 采集环境温度与湿度数据

校验机制: 每笔数据附带 CRC 校验, 校验通过才更新全局环境变量

采样频率: 主循环每 2 秒读取一次, 避免 I2C 总线过载

量程范围: 温度 -45℃~125℃, 湿度 0%~100%

核心函数: read_sht3x()

4.2.3 S7 人体存在传感器驱动

控制芯片: PCA9557 IO 扩展芯片

功能: 检测前方是否有人存在, 输出 0 (无人) / 1 (有人) 二元状态

核心函数: I2c_human_detected()

4.3 执行器驱动模块

4.3.1 E1 显示模块

包含 RGB 状态灯与 4 位数码管两部分, 均通过 I2C 总线控制。

(1) RGB 灯子模块

驱动芯片: PCA9685 PWM 控制器

支持颜色: 黑、红、橙、黄、绿、青、蓝、紫、白共 9 种预设颜色

亮度调节: 0~100% 线性调节, 通过缩放 RGB 分量实现

核心函数: e1_led_ctrl() (设置颜色 + 亮度)、e1_rgb_color_control() (直接控制 RGB 分量)

(2) 数码管子模块

驱动芯片: HT16K33

支持字符: 0~9、A~F、以及 t、H、-、_、. 等特殊符号

显示能力: 4 位独立控制, 支持数字、符号混合显示

核心函数: e1_digital_display() (4 位同步显示)、e1_digital_bit_display_char() (单字符显示)

4.3.2 E2 风扇模块

驱动芯片: PCA9685 PWM 控制器

控制精度: 0~100% 无级调速, 满占空比对应 0xFF

档位预设：0%、20%、40%、60%、80%、100% 共 6 档

核心函数：e2_speed_control()

4.4 E5 触控屏 Web 交互系统

(1) 架构设计

采用“C++ 后端生成状态 + Python HTTP 服务转发 + 前端轮询渲染”的解耦架构，避免 Web 逻辑阻塞主业务循环。

数据更新频率：前端每 800ms 轮询一次状态文件

指令传输：前端指令通过 HTTP 接口写入命令文件，C++ 主循环读取执行

(2) 后端 C++ 模块

状态生成：update_e5_display()函数每 50ms 更新一次 e5_status.json，包含实时环境数据、设备状态、历史曲线、语音历史等全量信息

指令处理：process_e5_command()函数读取 e5_command.txt，解析风扇、场景、语音、阈值等 12 类控制指令

历史数据：环形缓冲区存储最近 100 条环境记录，每秒写入一次，平衡数据连续性与 IO 负载

(3) Python HTTP 服务 (server.py)

基于标准库 http.server 实现，无需额外依赖；提供静态文件服务与 REST API 接口；所有前端 POST 指令转换为指定格式写入命令文件，与 C++ 后端解耦。

(4) 前端界面

双 Tab 布局：

仪表盘页：9 项核心状态卡片、环境变化趋势曲线图、告警信息、原始数据查看

控制中心页：风扇控制、灯光控制、场景切换、语音助手、阈值设置、休眠配置 6 大功能组

图表实现：基于 Chart.js 绘制温度、湿度、光照三条趋势曲线

交互逻辑：所有操作通过 sendCommand()函数异步发送，操作后 300ms 自动刷新状态



图 5: E5 Web 仪表盘界面布局示意图



图 6: E5 Web 控制中心界面布局示意图

4.5 音频与语音交互系统

4.5.1 音频播放子系统

独立后台线程，通过条件变量 + 互斥锁实现非阻塞播放；支持多请求排队，避免音频重叠播放；播放底层调用 aplay 工具，兼容 WAV 格式音频

4.5.2 TTS 语音合成

双模式降级机制：网络在线时优先使用小米云端 TTS，失败或离线时自动降级本地 TTS

本地引擎适配：自动检测 pico2wave、espeak-ng、espeak、flite 四套引擎，按优先级顺序 fallback

核心函数：tts_speak()（统一入口，自动选择 TTS 引擎）

4.5.3 全链路语音交互

完整流程：唤醒触发 → 3 秒同步录音 → 语音识别 → 语义解析 → 指令执行 → TTS 语音反馈

唤醒方式：按键 2 长按、E5 界面点击唤醒按钮

ASR 识别：在线时调用小米 ASR 接口，离线时跳过识别直接使用本地关键词匹配

NLP 解析：本地关键词匹配优先，覆盖 12 类核心意图；未匹配时调用云端 NLP 补充

意图覆盖：风扇控制、灯光控制、环境播报、场景切换、天气查询、日期时间查询等

对话历史：环形缓冲区存储最近 10 条交互记录，同步显示到 E5 界面

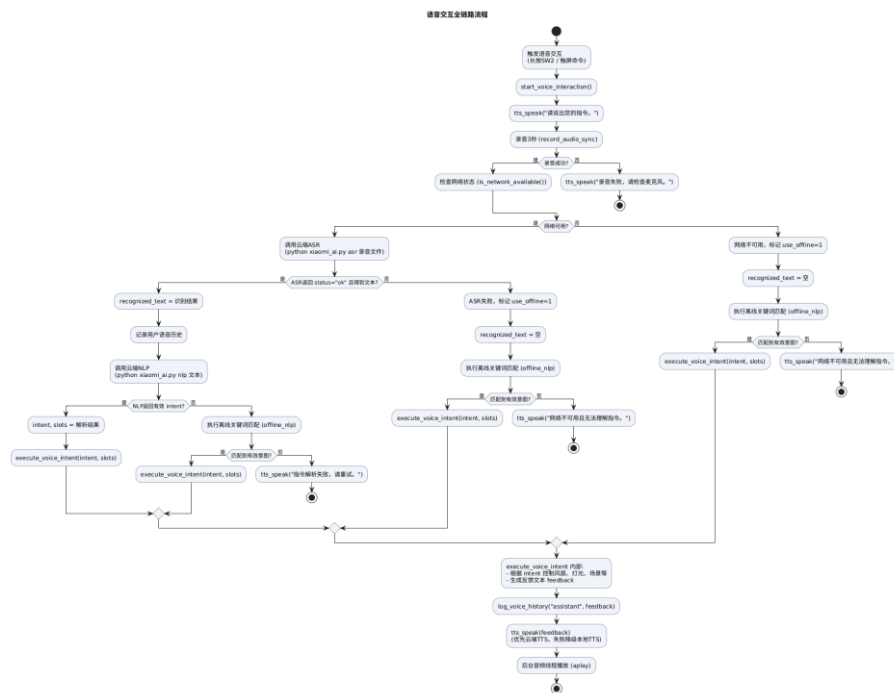


图 7：语音交互全链路流程图

4.6 智能联动引擎

(1) 场景化控制策略

内置工作、休闲、睡眠 3 种场景，不同场景对应差异化的风扇、灯光、休眠、告警策略。

场景模式	风扇控制策略	休眠阈值	告警机制	语音策略
工作模式	自动分档：<26℃关闭，26~28℃ 40%，28~30℃ 60%，≥30℃ 80%	无人 3 分钟	环境异常语音告警	正常播报
休闲模式	默认 20% 低档位，手动操作优先	无人 5 分钟	仅灯光提示	正常播报
睡眠模式	默认关闭，温度>35℃强制开启 60%	无人 1 分钟	仅灯光提示	完全禁用

表 4：场景控制策略表

(2) 其他联动规则

灯光自动调节：灯光自动模式下，根据光照强度线性调节 RGB 亮度

光照<100lx：亮度 20%

光照>1000lx：亮度 100%

中间区间：线性插值计算

人体感应休眠：无人超时后自动熄灭 E1 显示、关闭风扇、保存状态；检测到有人后自动恢复原有状态

睡眠模式唤醒后自动切换为休闲模式，避免打扰；

手动覆盖机制：用户通过按键 / 触控手动调节风扇后，标记为手动覆盖状态，自动模式暂停；切换场景后自动清除覆盖标记，恢复自动控制

核心实现函数：smart_control()

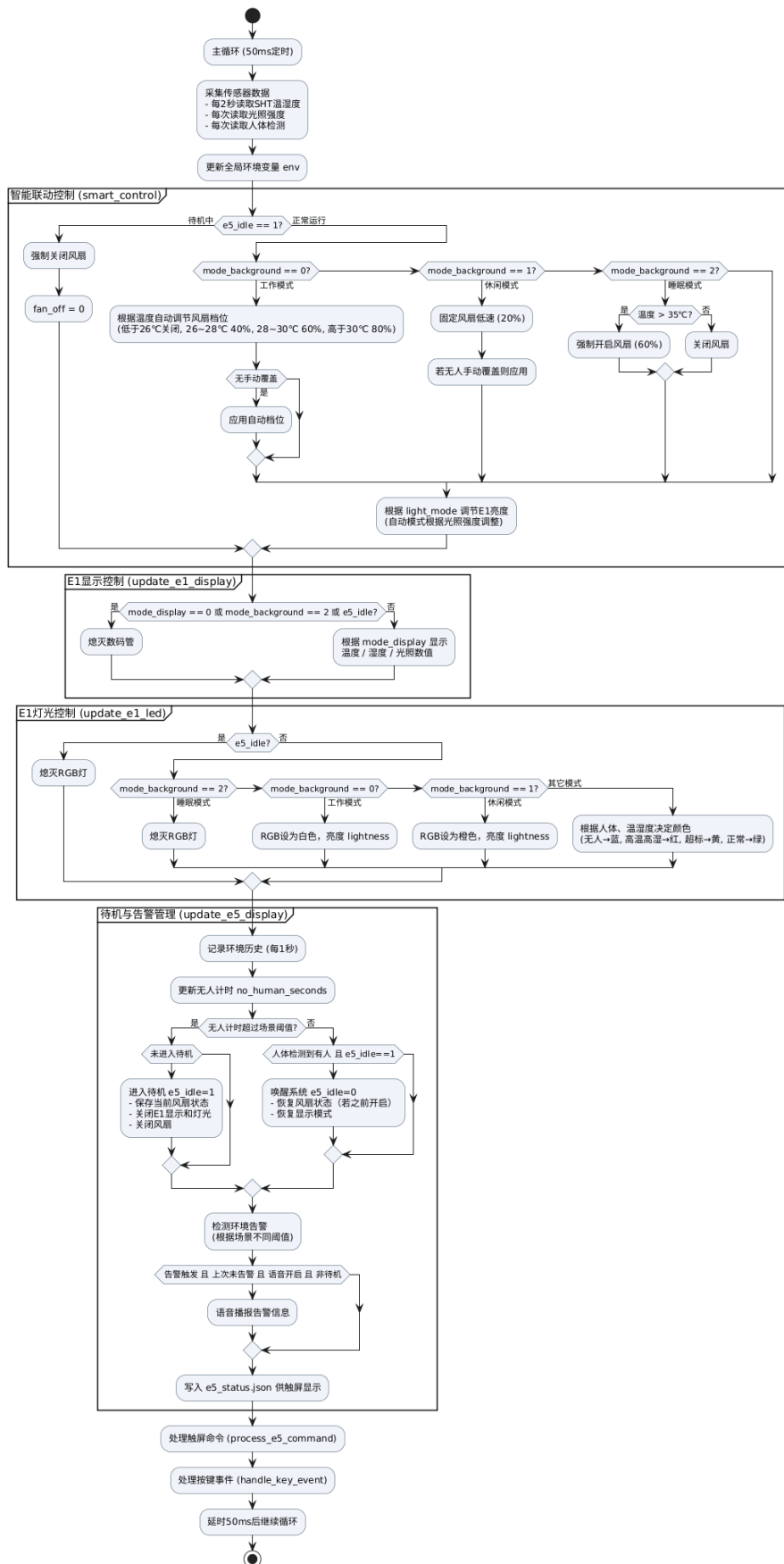


图 8：智能联动逻辑框图

4.7 网络监控与容错模块

独立监控线程：后台线程每 30 秒检测一次网络连通性，通过连接 8.8.8.8:53 公共 DNS 实现

线程安全：使用互斥锁保护网络状态全局变量，避免多线程读写竞争

全系统同步：网络状态实时同步到 E5 界面、语音交互系统、TTS 模块

离线降级逻辑：

语音识别：跳过云端 ASR，直接使用本地关键词匹配

TTS：自动切换为本地 TTS 引擎

天气查询：提示网络不可用，停止云端请求

核心实现函数：network_monitor_thread()、is_network_available()

五、接口定义与二次开发指南

5.1 新增 I2C 设备开发步骤

1. 在 i2c_probe.cpp 的 I2C_PROBE_LIST 数组中添加新设备的名称与候选地址列表
2. 新建对应头文件与源文件，按照规范实现 find（设备查找）、init（初始化）、读写控制三类接口
3. 在 main.cpp 的模块查找、初始化阶段添加新设备的调用代码
4. 在主循环中添加对应的数据采集或控制逻辑

5.2 新增语音意图开发步骤

离线规则：在 xiaomi_ai.py 的 nlp()函数中添加关键词与意图的映射关系

执行逻辑：在 main.cpp 的 execute_voice_intent()函数中新增对应 intent 的处理分支，实现业务逻辑

反馈优化：补充对应场景的 TTS 反馈文本，确保交互闭环

5.3 新增 Web API 开发步骤

前端：在 index.html 中添加交互控件，在 refresh.js 中绑定点击事件，调用 sendCommand()发送请求

服务端：在 server.py 的 do_POST()方法中新增 API 路径，将前端参数转换

为命令格式写入 e5_command.txt

后端：在 main.cpp 的 process_e5_command() 函数中新增命令解析分支，执行对应控制逻辑

5.4 新增智能联动场景开发步骤

1. 扩展 mode_background 全局变量的取值范围
2. 在 smart_control() 函数中添加新场景对应的风扇、灯光、休眠策略
3. 在按键、Web 界面的场景切换逻辑中添加新场景的选项

六、配置文件说明

6.1 xiaomi_config.json

小米 AI 平台配置文件，位于 web/ 目录下：

```
01  {
02      "BASE_URL": "https://api.xiaomimimo.com/v1",
03      "API_KEY": "用户专属 API 密钥",
04      "TTS_MODEL": "mimo-v2.5-tts",
05      "TTS_VOICE": "mimo_default",
06      "DEFAULT_CITY": "天津"
07  }
```

字段	说明
BASE_URL	小米 AI 开放平台服务地址
API_KEY	接口鉴权密钥，需自行申请替换
TTS_MODEL	语音合成使用的模型
TTS_VOICE	语音合成音色
DEFAULT_CITY	天气查询默认城市

表 5：xiaomi_config.json 字段说明表

6.2 运行时状态文件

文件名	路径	作用
e5_status.json	web/	全量系统状态文件，前端轮询读取
e5_command.txt	web/	前端指令临时文件，主循环读取后删除
e5_ui.txt	web/	精简状态文件，供旧版前端快速读取
voice_history.json	web/	语音交互历史记录持久化存储

表 6：运行时状态文件功能表

七、常见问题与调试方法

- (1) I2C 设备探测失败
 - 1. 检查子板是否正确插入母板对应槽位
 - 2. 确认设备候选地址与硬件实际地址一致
 - 3. 查看系统 /dev 目录下是否存在对应 I2C 总线设备节点
- (2) 按键误触发 / 响应不灵敏
 - 1. 调整 s1.h 中的 DEBOUNCE_TIME 消抖时间，误触发可适当增大
 - 2. 调整 LONG_PRESS_TIME 长按阈值，优化长短按判定边界
- (3) E5 Web 界面无法访问
 - 1. 确认 server.py 已正常启动，端口未被占用
 - 2. 检查开发板与访问设备是否在同一网段
 - 3. 查看防火墙是否拦截对应端口
- (4) 语音交互无响应
 - 1. 检查网络连通性与 API 密钥是否有效
 - 2. 确认麦克风设备正常，录音权限开启
 - 3. 离线模式下检查指令是否在本地关键词匹配范围内
- (5) 智能联动不生效
 - 1. 确认当前处于对应场景模式
 - 2. 检查是否处于手动覆盖状态，切换场景可重置
 - 3. 确认传感器数据采集正常，环境参数符合触发条件